# EE 789

## RTL Design of Packet Switch Architectures

Submitted by

Athav Vantan B

Roll Number: 23B1306

# Contents


**1 Introduction** **2**

**2 Packet Format** **2**

**3 Experimental Setup and Testing Methodology** **3**
3.1 Simulation Flow . . . . . 3

**4 Shared Queue Switch** **3**
4.1 Architecture Overview . . . . . 3
4.2 Input Arbitration Strategy . . . . . 4
4.3 Packet Storage . . . . . 5
4.4 Output Routing Logic . . . . . 5
4.5 Algorithm . . . . . 5
4.6 Design Discussion . . . . . 6

**5 Input Queued Switch** **6**
5.1 Architecture Overview . . . . . 6
5.2 Input Handling . . . . . 7
5.3 Output Arbitration Using Finite State Machines . . . . . 8
5.4 Conflict Resolution . . . . . 8
5.5 Packet Transmission . . . . . 8
5.6 Algorithm . . . . . 9
5.7 Design Discussion . . . . . 9

**6 Output Queued Switch** **10**
6.1 Architecture Overview . . . . . 10
6.2 Input Demultiplexing . . . . . 11
6.3 Packet Storage . . . . . 11
6.4 Output Arbitration . . . . . 11
6.5 Packet Transmission . . . . . 12
6.6 Algorithm . . . . . 12
6.7 Design Discussion . . . . . 13

**7 Traffic Test Scenarios** **13**
7.1 Both Inputs to Both Outputs . . . . . 13

**8 Comparison of Switch Architectures** **14**

**9 Conclusion** **15**

# 1 Introduction

Packet switches are fundamental components used in communication networks to route data packets from multiple sources to their intended destinations. A switch receives packets from its input ports and forwards them to the appropriate output ports based on routing information contained in the packet header.

In this assignment, a $2 \times 2$ packet switch was designed and implemented using Register Transfer Level (RTL) design in VHDL. The switch consists of two input ports and two output ports. Incoming packets may arrive at either input port and must be forwarded to one of the output ports based on the destination field contained in the packet header.

The goal of this assignment is to study and implement different architectural approaches for packet switching. Three switch architectures were implemented:

- Shared Queue Switch
- Input Queued Switch
- Output Queued Switch

Each architecture differs in the way packets are buffered and how contention between inputs and outputs is handled. The designs were implemented in VHDL and validated using the testing infrastructure provided with the assignment bundle.

# 2 Packet Format

Packets entering the switch consist of multiple 32-bit words. The first word of every packet acts as the header and contains routing information used by the switch.

| Bits | Interpretation |
|---|---|
| [31:24] | Destination |
| [23:8] | Length |
| [7:0] | Sequence-id |

Figure 1: Header Description

The destination field specifies which output port the packet must be sent to. The packet length field indicates the total number of words in the packet. The sequence identifier uniquely identifies packets within the incoming traffic stream.

When a packet arrives at the switch, the header is examined to determine the correct output port. The switch then forwards the entire packet to the corresponding destination.

# 3 Experimental Setup and Testing Methodology

All switch architectures were implemented in VHDL using the bundle framework provided with the assignment. The design files were placed inside the `vhdl` directory of the bundle.

The implemented modules are listed below:

- `shared_queue_switch.vhd`
- `input_queue_switch.vhd`
- `output_queue_switch.vhd`

In addition to these modules, the queue implementation provided with the assignment (`Queue.vhd`) was used directly. This queue module implements a parameterized FIFO structure and provides the necessary enqueue and dequeue functionality required for buffering packet data.

## 3.1 Simulation Flow

The designs were compiled and tested using the AHIR Docker environment provided with the assignment.

The testing procedure is summarized below.

- The switch architecture to be tested is selected by modifying the file `build_ghdl_model.sh` located in the bundle directory.
- The bundle directory is mounted inside the AHIR Docker container.
- The build process is initiated using the command:

```
./build.sh
```

- After compilation, the simulation environment is launched.
- The testbench is executed and monitored using `tmux` sessions to manage multiple terminal windows.
- The same procedure is repeated for each of the three switch architectures by selecting the corresponding VHDL file in the build script.

This approach allows the same testing infrastructure and testbench environment to be reused across all switch architectures, ensuring consistent validation of the implemented designs.

# 4 Shared Queue Switch

## 4.1 Architecture Overview

The shared queue switch architecture uses a single centralized queue to buffer packets arriving from multiple input ports. All packets entering the switch are first inserted into this shared queue before being forwarded to the appropriate output port.

Figure 2 illustrates the conceptual architecture of the shared queue switch.

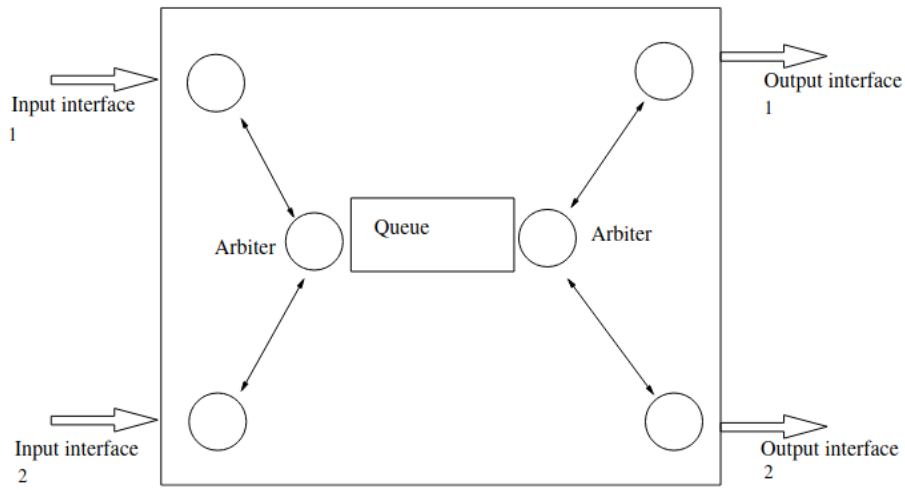


Figure 2: Shared Queue Switch Architecture

The main components of the shared queue switch are:

- Two input interfaces
- A shared FIFO queue
- Input arbitration logic
- Output routing logic
- Two output interfaces

Since both input ports may attempt to write into the queue at the same time, an arbitration mechanism is required to determine which input port is allowed to enqueue data.

## 4.2 Input Arbitration Strategy

Each input interface provides incoming data along with a `valid` signal indicating that the data word is ready to be transmitted into the switch.

The arbitration logic operates as follows.

- If only one input port asserts its `valid` signal, that input is granted immediate access to the queue.
- If both input ports assert the `valid` signal simultaneously, the arbiter determines which input should be served first.
- A variable called `last_served` is maintained to record which input port was granted access during the previous arbitration cycle.

- When both inputs request access simultaneously, priority is given to the input port that was not served in the previous cycle.

This alternating arbitration mechanism ensures fairness between the two input ports and prevents starvation.

## 4.3 Packet Storage

Once an input port is granted access by the arbiter, the incoming data words are written into the shared queue. Since packets consist of multiple words, the entire packet is stored sequentially in the queue.

The first word of the packet contains the header, which includes the destination field, packet length, and sequence identifier. This information is later used by the output logic to route the packet to the correct destination.

## 4.4 Output Routing Logic

When a packet is dequeued from the shared queue, the destination field in the packet header is examined.

Based on this destination value, the packet is routed to the appropriate output interface.

Once transmission of a packet begins, the output port remains reserved until the entire packet has been transmitted. This ensures that packets are delivered atomically and prevents interleaving of packet data from multiple packets.

## 4.5 Algorithm

The high-level algorithm implemented by the shared queue switch is shown below.

```
Input Arbitration

if input1_valid and not input2_valid:
grant input1

else if input2_valid and not input1_valid:
grant input2

else if input1_valid and input2_valid:
if last_served == input1:
grant input2
else:
grant input1

enqueue selected input into shared queue
```

```
Output Processing

read header from queue
extract destination field

if destination == output1:
send packet to output1

else:
send packet to output2

lock output port
transmit entire packet
release output port
```

### 4.6 Design Discussion

The shared queue architecture simplifies buffer management by maintaining a single centralized queue for all incoming packets. This approach reduces the complexity of the buffering structure and allows packets from different input ports to be stored in the same location.

However, because multiple inputs share the same queue, arbitration logic is required to ensure fair access. The implemented alternating arbitration scheme provides a simple and effective mechanism to avoid starvation.

Once packets are stored in the queue, routing becomes straightforward since packets are processed sequentially and forwarded to the appropriate output port based on the destination field contained in the packet header.

## 5 Input Queued Switch

### 5.1 Architecture Overview

In the input queued switch architecture, each input port maintains its own dedicated queue. Instead of storing all incoming packets in a single shared buffer, packets arriving from each input are stored in separate queues associated with the respective input ports.

Figure 3 illustrates the conceptual architecture of the input queued switch.

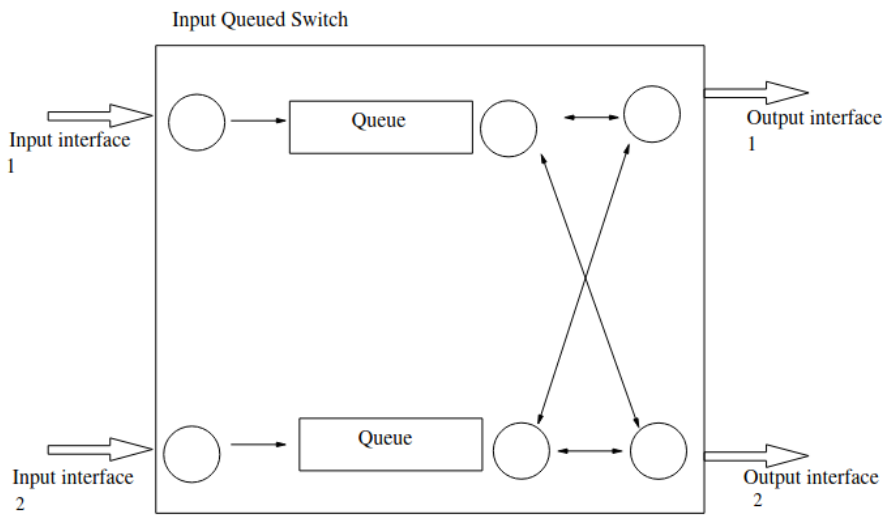


Figure 3: Input Queued Switch Architecture

The primary components of this architecture are:

- Two input interfaces
- Two independent input queues
- Output arbitration logic
- Two output interfaces

Each input interface writes incoming packet data into its corresponding queue. When packets need to be transmitted to the outputs, arbitration logic determines which queue is allowed to send data to the requested output port.

## 5.2 Input Handling

The input side of the input queued switch is relatively straightforward. Each input port has a dedicated queue associated with it.

The logic implemented for the input side operates according to the following rules:

- When an input port has valid data available, it attempts to push the data into its corresponding queue.
- The data is written into the queue only if the queue is not full.
- If the queue is full, the input interface must wait until space becomes available.

Since each input has its own queue, there is no contention between inputs during the enqueue operation. This simplifies the input side of the switch compared to the shared queue architecture.

## 5.3 Output Arbitration Using Finite State Machines

The output side of the switch requires arbitration because packets from both input queues may request the same output port simultaneously.

To handle this, two independent finite state machines (FSMs) are implemented. Each FSM manages packet transmission from one of the input queues.

The FSMs perform the following tasks:

- Inspect the header of the packet at the head of the queue.
- Determine the destination output port from the destination field.
- Request access to the required output port.
- Transmit the packet once the output port becomes available.

Two output ports exist in the system, therefore the FSM must decide whether the packet should be routed to output port 1 or output port 2.

## 5.4 Conflict Resolution

When both input queues attempt to send packets to the same output port simultaneously, arbitration is required.

To ensure fairness between the two input queues, a variable called `last_served` is used. This variable records which input queue was granted access to the output port during the previous arbitration event.

The arbitration mechanism works as follows:

- If only one queue requests a particular output port, that queue is granted access immediately.
- If both queues request the same output port simultaneously, the queue that was not served in the previous arbitration cycle is given priority.
- The `last_served` variable is then updated to reflect the newly served queue.

This approach implements a simple round-robin style arbitration scheme that prevents starvation.

## 5.5 Packet Transmission

Once an FSM successfully acquires the required output port, packet transmission begins.

The packet is transmitted word by word until the entire packet has been sent. A packet length counter is used to track how many words remain in the packet.

During packet transmission:

- The output port is marked as busy.

- No other packet is allowed to use the output port until the current packet finishes transmission.
- Once the packet transmission completes, the FSM returns to the idle state and begins evaluating the next packet in the queue.

This ensures that packets are transmitted atomically and prevents interleaving of packet data.

## 5.6 Algorithm

The high-level algorithm implemented by the input queued switch is summarized below.

```
Input Side:

if input_valid and queue_not_full:
push data into corresponding input queue

Output Side:

for each input queue:

```
read packet header

determine destination output port

if output port is free:
    transmit packet

else if both queues request same output:
    use last_served variable to decide priority

lock output port
send entire packet
release output port
```
```

## 5.7 Design Discussion

The input queued architecture separates the buffering structure for each input port, which eliminates contention between inputs during packet arrival. This allows packets to be accepted from both inputs simultaneously as long as their respective queues have space available.

However, contention may still occur at the output stage when multiple input queues attempt to send packets to the same output port. The arbitration logic implemented using finite state machines resolves this contention while ensuring fairness between input queues.

This architecture improves scalability compared to the shared queue design while maintaining relatively simple control logic.

# 6 Output Queued Switch

## 6.1 Architecture Overview

The output queued switch architecture differs from the previous designs in that packet buffering is organized according to the destination output ports. Instead of storing packets in a shared buffer or per-input queues, packets are directed into queues associated with their destination output ports.

In this design, packets arriving at the input interfaces are first examined to determine their destination port. Based on this information, the packet is routed to the appropriate destination queue through a demultiplexing stage.

Figure 4 illustrates the architecture of the output queued switch.

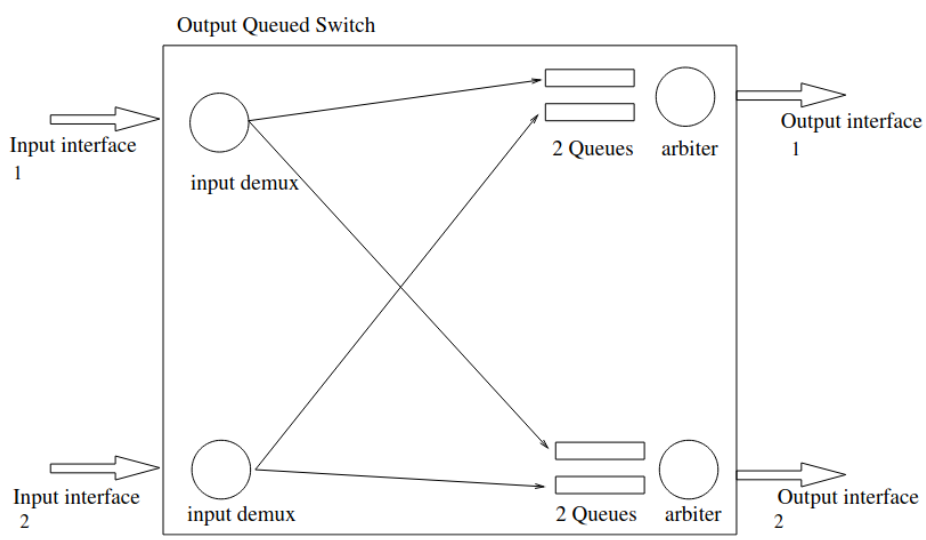


Figure 4: Output Queued Switch Architecture

To support simultaneous packet arrivals from two inputs and routing to two outputs, four queues are used:

- Queue for packets from Input 1 destined to Output 1
- Queue for packets from Input 1 destined to Output 2

- Queue for packets from Input 2 destined to Output 1
- Queue for packets from Input 2 destined to Output 2

This organization is commonly referred to as a **virtual output queue** structure, where each input maintains a separate queue for each possible destination.

## 6.2 Input Demultiplexing

When packets arrive at the input ports, the destination field in the packet header is examined to determine which output port the packet must be sent to.

A demultiplexing stage routes incoming packet data to the correct destination queue. Two finite state machines (FSMs) are used to control this process, one for each input port.

The FSM associated with each input port performs the following tasks:

- Inspect the header of the incoming packet.
- Determine the destination output port.
- Route the packet data to the corresponding destination queue.
- Track the remaining packet length to ensure the entire packet is routed correctly.

Packet boundaries are maintained using a length counter derived from the packet header. Once the header is received, the length field specifies how many additional words belong to the packet. The FSM continues routing data to the selected queue until the packet transmission is complete.

## 6.3 Packet Storage

The demultiplexed packets are stored in the appropriate destination queues. Each queue behaves as a FIFO buffer that stores packet data until the output interface is ready to transmit it.

Using separate queues for each input-output pair ensures that packets destined for different outputs do not block each other. This structure allows packets destined for different outputs to be processed independently.

## 6.4 Output Arbitration

Each output port may receive packets from two different queues, corresponding to the two input ports. Therefore, arbitration is required to determine which queue should be served when both queues contain packets.

To resolve this contention, an arbiter is implemented for each output port. The arbiter alternates between the two queues in order to ensure fairness.

A variable called `last_served` is maintained for each output port to track which queue was served previously. When both queues contain packets, the arbiter selects the queue that was not served in the previous arbitration cycle.

This alternating selection mechanism ensures that neither input is permanently favored and that both queues receive fair access to the output port.

## 6.5 Packet Transmission

Once a queue is selected by the arbiter, packet transmission begins. The entire packet is transmitted sequentially before another queue is allowed to access the output port.

During packet transmission:

- The output port is reserved for the selected queue.
- Packet data is transmitted word by word.
- A packet length counter tracks the remaining words in the packet.
- When the packet transmission completes, the arbiter returns to the idle state and selects the next queue to serve.

This approach ensures that packets are delivered atomically and prevents interleaving of data from different packets.

## 6.6 Algorithm

The high-level algorithm implemented by the output queued switch is summarized below.

```
Input Demultiplexing:

for each input port:

'''
read packet header
determine destination output port

if destination == output1:
    route packet to queue (input -> output1)

else:
    route packet to queue (input -> output2)

continue routing until packet length completes
'''

Output Arbitration:
```

```
for each output port:

'''
if both queues contain packets:
    select queue using last_served variable
else
    select the queue that contains data

transmit packet
lock output port during transmission
release port after packet completes
'''
```

### 6.7 Design Discussion

The output queued switch architecture provides improved isolation between traffic flows by separating queues based on both input and output ports. This reduces contention and allows packets destined for different outputs to progress independently.

The use of demultiplexing logic at the input stage ensures that packets are directed to the correct destination queues as soon as they arrive. Arbitration at the output stage ensures fair access between the queues that feed each output port.

Although this architecture requires more buffering resources compared to the shared queue and input queued designs, it provides better performance under contention and more predictable packet forwarding behavior.

## 7 Traffic Test Scenarios

To validate the correctness of the switch implementations, the designs were tested using the provided testbench. The objective of these tests was to ensure that packets were correctly routed to their intended outputs and that the arbitration logic behaved correctly when multiple inputs attempted to access the same output.

### 7.1 Both Inputs to Both Outputs

In this scenario, packets are generated from both input ports and are destined for both output ports. This represents a more comprehensive traffic pattern where packets from each input may request different outputs at different times.

The provided testbench was used to simulate this behavior and verify that the switch correctly routes packets to the appropriate outputs while handling contention when necessary. The results confirm that the implemented arbitration and buffering mechanisms allow packets from both inputs to be delivered correctly to both outputs.

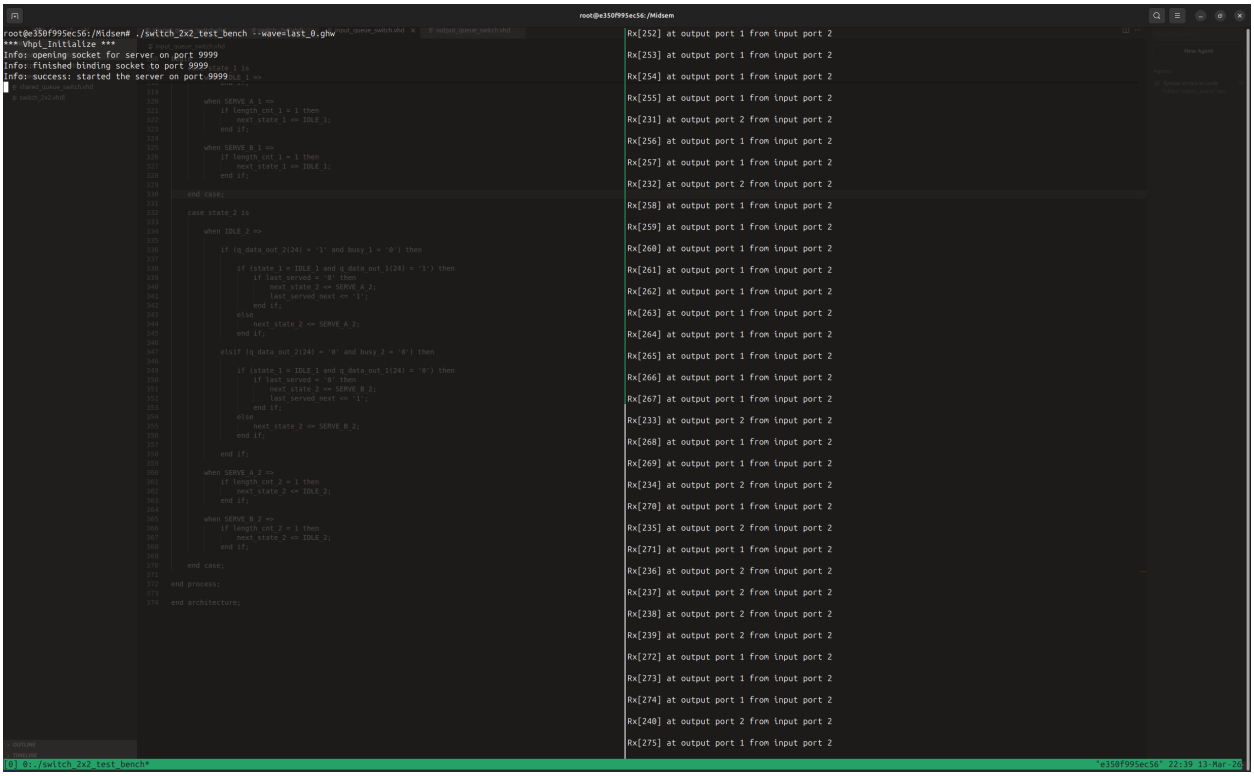


Figure 5: Simulation result showing both inputs successfully transmitting packets to both outputs using the provided testbench.

# 8 Comparison of Switch Architectures

The three switch architectures implemented in this assignment differ in the way packets are buffered and how contention is handled.

- **Shared Queue Switch** uses a single centralized queue for all incoming packets. This simplifies buffer management but requires arbitration when multiple inputs attempt to access the queue simultaneously.
- **Input Queued Switch** maintains a separate queue for each input port. This allows packets to arrive simultaneously without contention at the input stage, but conflicts may still occur at the output ports when multiple inputs request the same destination.
- **Output Queued Switch** organizes buffers based on output destinations using virtual output queues. Packets are directed to the appropriate queue immediately after arrival, reducing contention between different traffic flows.

Each architecture presents different trade-offs between implementation complexity, buffering requirements, and arbitration logic.

# 9 Conclusion

In this assignment, three different architectures for implementing a $2 \times 2$ packet switch were designed and implemented using RTL in VHDL. The shared queue, input queued, and output queued switch designs demonstrate different approaches to handling packet buffering and arbitration.

The shared queue architecture provides a simple buffering structure but requires arbitration at the input stage. The input queued architecture separates buffering across inputs but requires output arbitration when multiple queues request the same output port. The output queued architecture further improves isolation between traffic flows by organizing queues based on destination outputs.

All designs were successfully implemented and tested using the provided simulation framework. The results demonstrate correct packet routing and proper functioning of the arbitration mechanisms under different traffic scenarios.